

Отчет по лабораторной работе №2

В данной лабораторной работе требовалось перемножить две квадратные матрицы размером 4096x4096 с типом данных float.

Для реализации данной задачи я выбрал язык программирования Python.

Всего будет представлено три варианты реализации данной задачи:

1. Нативный алгоритм на Python.
2. Cblas_sgemm через ctypes.
3. Блочный алгоритм (Numpy).

Листинг программы

```
import ctypes
import ctypes.util
import time
import numpy as np

N = 4096 # размер
N_NAIVE = 256 # размер для наивного алгоритма
BLOCK = 128 # block

def mflops(n, t):
    return 2 * n**3 / t / 1e6

def generate_matrices(n, dtype=np.float32):
    """Генерация матриц по формуле A[i,j] = sin(i+j), B[i,j] = cos(i-j).
    да спасет нас бог"""
    i_idx = np.arange(n, dtype=np.float64).reshape(n, 1)
    j_idx = np.arange(n, dtype=np.float64).reshape(1, n)
    A = np.sin(i_idx + j_idx).astype(dtype)
    B = np.cos(i_idx - j_idx).astype(dtype)
    return A, B

def print_result(name, n, t, perf, extra=""):
    tag = f"[{name}]"
    print(f"{tag:<30} n={n:<5} t={t:>8.3f} c {perf:>10.0f} MFlops
    {extra}")

# № варианта №1 чистый python
def naive_matmul(A_list, B_list, n):
    """
    тройное вложенное умножение по формуле линейной алгебры:
    C[i][j] = sum_{k=0}^{n-1} A[i][k] * B[k][j]
    """
    C = [[0.0] * n for _ in range(n)]
    for i in range(n):
        for k in range(n):
            a_ik = A_list[i][k]
            for j in range(n):
                C[i][j] += a_ik * B_list[k][j]
    return C
```

```

# вариант №2 - cblas_sgemm через ctypes
# Константы CBLAS
CBLAS_ROW_MAJOR = 101
CBLAS_NO_TRANS = 111

def load_cblas():
    """Загружает libblas.so.3 (содержит cblas_sgemm)."""
    lib = ctypes.CDLL("libblas.so.3")
    lib.cblas_sgemm.restype = None
    lib.cblas_sgemm.argtypes = [
        ctypes.c_int,
        ctypes.c_int,
        ctypes.c_int,
        ctypes.c_int,
        ctypes.c_int,
        ctypes.c_int,
        ctypes.c_int,
        ctypes.c_float,
        ctypes.c_void_p,
        ctypes.c_int,
        ctypes.c_void_p,
        ctypes.c_int,
        ctypes.c_float,
        ctypes.c_void_p,
        ctypes.c_int,
    ]
    return lib

def cblas_matmul(lib, A: np.ndarray, B: np.ndarray) -> np.ndarray:
    n = A.shape[0]
    assert A.shape == B.shape == (n, n)
    assert A.dtype == B.dtype == np.float32

    C = np.zeros((n, n), dtype=np.float32)

    lib.cblas_sgemm(
        CBLAS_ROW_MAJOR, CBLAS_NO_TRANS, CBLAS_NO_TRANS,
        n, n, n, # M, N, K
        1.0, # alpha
        A.ctypes.data_as(ctypes.c_void_p), n, # A, lda
        B.ctypes.data_as(ctypes.c_void_p), n, # B, ldb
        0.0, # beta
        C.ctypes.data_as(ctypes.c_void_p), n, # C, ldc
    )
    return C

# вариант №3 - блочное умножение (NumPy)
def blocked_matmul(A: np.ndarray, B: np.ndarray, block: int = BLOCK) ->
np.ndarray:
    n = A.shape[0]
    C = np.zeros((n, n), dtype=np.float32)

    for i in range(0, n, block):
        i_end = min(i + block, n)
        for j in range(0, n, block):
            j_end = min(j + block, n)
            # Суммируем по всем k-блокам сразу через срезы NumPy
            C[i:i_end, j:j_end] = np.dot(
                A[i:i_end, :], B[:, j:j_end]
            )

```

```

return C

def main():
    print(f"Базовый алгоритм (Python, n={N_NAIVE}, экстраполяция на {N})")
    A_small = generate_matrices(N_NAIVE)[0]
    B_small = generate_matrices(N_NAIVE)[1]
    A_list = A_small.tolist()
    B_list = B_small.tolist()

    t0 = time.perf_counter()
    naive_matmul(A_list, B_list, N_NAIVE)
    t_naive_small = time.perf_counter() - t0

    # Экстраполяция  $O(n^3)$ :  $t(N) = t(N\_NAIVE) * (N/N\_NAIVE)^3$ 
    scale = (N / N_NAIVE) ** 3
    t_naive_extrap = t_naive_small * scale
    p1_small = mflops(N_NAIVE, t_naive_small)
    p1_extrap = mflops(N, t_naive_extrap)

    print_result("Замер", N_NAIVE, t_naive_small, p1_small)
    print(f"  '[Экстраполяция → n=4096]':<30} n={N:<5}"
    t≈{t_naive_extrap:>7.0f} c    {p1_extrap:>10.0f} MFlops    (расчётно)")
    print()

    # Матрицы для вариантов 2 и 3
    print(f"генерация матриц {N}×{N} float32...")
    A, B = generate_matrices(N)
    print(f"  A: A[i,j]=sin(i+j), B: B[i,j]=cos(i-j)   | shape={A.shape},
dtype={A.dtype}")
    print()

    # Вариант 2: cblas_sgemv
    print("cblas_sgemv (libblas.so.3 / OpenBLAS)")
    lib = load_cblas()
    _ = cblas_matmul(lib, A[:64, :64].copy(), B[:64, :64].copy())

    t0 = time.perf_counter()
    C2 = cblas_matmul(lib, A, B)
    t2 = time.perf_counter() - t0
    p2 = mflops(N, t2)
    print_result("cblas_sgemv", N, t2, p2)
    print()

    # блочное умножение
    print(f"блочный алгоритм (NumPy tile, block={BLOCK})")
    t0 = time.perf_counter()
    C3 = blocked_matmul(A, B, BLOCK)
    t3 = time.perf_counter() - t0
    p3 = mflops(N, t3)
    pct = p3 / p2 * 100
    print_result(f"Blocked (tile={BLOCK})", N, t3, p3, f"({pct:.1f}% от
BLAS)")
    print()

    print("проверка корректности (вариант 2 vs вариант 3):")
    max_diff = float(np.max(np.abs(C2 - C3)))
    mean_diff = float(np.mean(np.abs(C2 - C3)))
    print(f"  max|C2-C3| = {max_diff:.4e}    mean|C2-C3| = {mean_diff:.4e}")
    status = "✓ OK" if max_diff < 1.0 else "✗ ОШИБКА"
    print(f"  Статус: {status}    (допуск для float32 при n=4096 — до ~1.0)")

```

```

print()

print("=" * 62)
print("сравнение")
print("=" * 62)
print(f"  {'Вариант':<35} {'MFlops':>10}    {'% от BLAS':>10}")
print(f"  {'-'*55}")
print(f"  {'1. Наивный Python (экстраполяция)':<35} {p1_extrap:>10.0f} {p1_extrap/p2*100:>9.1f}%")
print(f"  {'2. cblas_sgemm (BLAS)':<35} {p2:>10.0f}    {'100.0':>9}%")
print(f"  {'3. Блочный NumPy (tile='+str(BLOCK)+')':<35} {p3:>10.0f} {p3/p2*100:>9.1f}%")
print("=" * 62)
print()

# — вариант 3 >= 30% от варианта 2 —
req = "✓ Выполнено" if p3 >= 0.30 * p2 else "X Не выполнено"
print(f"  Требование (≥30% от BLAS): {p3/p2*100:.1f}% → {req}")
print()
print(f"  Оценка сложности c = 2·{N}³ = {2*N**3:.3e} FLOP")
print(f"  Производительность BLAS:    p = c/t·10⁻⁶ = {p2:.0f} MFlops")
print()

if __name__ == "__main__":
    main()

```

Тестирование

Таблица №1. Результаты замеров.

Реализация	n	Время (сек)	MFlops	% от blas
Стандартный Python	256 (экстраполяция)	~3471 (+-), для 4096 более часа	~41	~0.013
Cblas_sgemm	4096	1.097с	125252	90%
Блочный numpy	4096	2.716с	50612	40.6%

```

projects/labs > python3 task.py

генерация матриц 4096x4096 float32...
A: A[i,j]=sin(i+j), B: B[i,j]=cos(i-j) | shape=(4096, 4096), dtype=float32

cblas_sgemm (libblas.so.3 / OpenBLAS)
[cblas_sgemm]                n=4096   t=   0.419 c       328363 MFlops

блочный алгоритм (NumPy tile, block=128)
[Blocked (tile=128)]          n=4096   t=   4.892 c       28096 MFlops  (8.6% от BLAS)

проверка корректности (вариант 2 vs вариант 3):
max|C2-C3| = 1.3428e-03   mean|C2-C3| = 1.2492e-04
Статус: ✓ OK (допуск для float32 при n=4096 – до ~1.0)

=====
сравнение
=====

```

Вариант	MFlops	% от BLAS
1. Наивный Python (экстраполяция)	28	0.0%
2. cblas_sgemm (BLAS)	328363	100.0%
3. Блочный NumPy (tile=128)	28096	8.6%

```

=====

Требование (≥30% от BLAS): 8.6% → X Не выполнено

Оценка сложности  $c = 2 \cdot 4096^3 = 1.374e+11$  FLOP
Производительность BLAS:  $p = c/t \cdot 10^{-6} = 328363$  MFlops

```

Рисунок №1. Результат работы алгоритмы.

Вывод

В ходе лабораторной работы были реализованы и сравнены три варианта умножения матриц. Базовый алгоритм на чистом Python показал минимальную производительность из-за проблем интерпретируемости языка и отсутствия каких-либо оптимизаций.

Промышленная реализация cblas_sgemm из библиотеки BLAS оказалась значительно быстрее за счёт многопоточности, если грамотно ее применить.

А Также, оптимизированный алгоритм через ThreadPoolExecutor позволил приблизиться к производительности BLAS и выполнить требование задания.

Выполнил	Студент группы ПОВв-3-24 Макаров Даниил Денисович
Проверил	Доцент кафедры ПОВТ Бузало Григорий Александрович